

Flight Controller Firmware

Development Report -- Part 2: HAL Realization, Vertical Navigation, and Final Verification

Author: Ansh Vishwakarma

Project: Quad-X Flight Controller Firmware with Host-Side Simulation

1. Overview

This document is a continuation of the Phase 1 development report. Part 1 covered the core attitude pipeline -- the Mahony estimator, cascade PID, quad-X mixer, and DShot encoder -- along with the debugging process that got those subsystems working correctly. This report documents what changed after that: completion of the Hardware Abstraction Layer, the vertical navigation stack (\$7.11 stretch goals), and the final performance verification against all required scenarios.

2. HAL Completion

The most significant architectural gap left at the end of Phase 1 was the sensor side of the HAL. The motor output was already fully decoupled -- `sim_main.c` called through `motor_mock.init()`, `motor_mock.arm()`, and `motor_mock.send_dshot()` without touching the plant model directly. The IMU and magnetometer were still calling `plant_generate_gyro()`, `plant_generate_accel()`, and `plant_generate_mag()` directly from `sim_main.c`, bypassing the `imu_dev_t` and `mag_dev_t` interface structs that already existed in `hal/`.

Closing this gap required three specific changes:

Routing sensor reads through the HAL structs. The `plant_generate_*` calls in `sim_main.c` were replaced with `imu_mock.read()` and `mag_mock.read()`, which call those same plant functions internally. The firmware in `src/` now has no direct reference to any plant function.

Fixing a hidden timestamp bug. While wiring this in, I found that `imu_mock.c` and

`mag_mock.c` were both hardcoding the sensor timestamp to a fixed value (1000) instead of using the live simulation clock. This meant the noise and bias model would have been evaluated at a frozen time rather than advancing with the simulation. The fix was to pass `current_sim_time_us` into both functions. The CSV output before and after this change is byte-for-byte identical -- confirming the rewiring changed the call path without altering any physics or results.

Adding the barometer HAL. A new `baro_dev_t` struct was defined in `hal/baro_hal.h`, following the same function-pointer pattern as `imu_hal.h` and `mag_hal.h`. The host implementation in `drivers_host/baro_mock.c` calls `plant_generate_baro()` internally, applying the standard hypsometric pressure model to the plant's ground-truth Z position with MS5611-class sensor noise added. This completes the HAL layer for all four sensor types.

3. Vertical Navigation Stack (§7.11)

3.1 Barometer Physics Model

`plant_generate_baro()` implements the standard atmosphere model. Given the drone's ground-truth altitude from the physics state, it computes the ambient pressure using:

$$P = P_0 \times (T / T_0)^{5.25588}$$

where T follows the standard lapse rate of 0.0065 K/m from the sea-level baseline of 288.15 K.

Gaussian noise of 2.0 Pa variance is added to the pressure output, and 0.05°C to the temperature -- representative of real MS5611 barometer noise characteristics.

3.2 Kalman Altitude Estimator

Raw barometric pressure is converted to altitude using the inverse hypsometric formula in `baro_to_altitude()`, then fed into a 1D Kalman filter implemented in `altitude.c`. The filter maintains two state variables -- estimated altitude z and estimated vertical velocity v -- with a 2x2 error covariance matrix updated each tick.

The predict step uses a constant-velocity kinematic model:

$$\begin{aligned} z_{\text{pred}} &= z + v \times dt \\ v_{\text{pred}} &= v \end{aligned}$$

The update step fuses the barometric altitude measurement with the prediction using the standard Kalman gain derivation. Process noise parameters `Q_pos` and `Q_vel` are tunable in `config.c`, as is the measurement noise `R_baro`. The filter is initialized from the first barometric reading at startup so the estimator begins with a physically correct baseline rather than an arbitrary zero.

3.3 Altitude Hold Controller

The altitude hold loop lives in `control_update_altitude()` in `control.c`, structured as a cascade matching the attitude controller:

Outer loop: altitude error $\rightarrow$ target climb rate, clamped to `ALT_MAX_CLIMB_RATE` (1.5 m/s).

Inner loop: climb rate error $\rightarrow$ throttle correction, using P and I terms with anti-windup clamping at $\pm$ `ALT_INTEGRAL_LIMIT`.

The derivative term is computed on the velocity measurement rather than the error, for the same reason the attitude rate loop uses derivative-on-measurement -- it avoids a sudden output kick when the setpoint changes. All gains and limits are declared in `config.c`.

In `fc_main.c`, altitude hold activates when the throttle stick sits between 0.45 and 0.55.

The correction output is added to a fixed feedforward hover throttle. When the stick moves outside this band, the integrator resets and the target altitude is updated to the current estimated altitude, so re-engaging altitude hold later does not produce a step correction from stale state.

3.4 Yaw Heading Hold

The yaw stick input is integrated each tick into a persistent `target_heading_rad` variable, which is wrapped into the $\pm\pi$ range to prevent unbounded accumulation. This value is passed directly as the yaw setpoint to `control_update()`. When the stick is centered, the target stops changing and the yaw PID holds the last commanded heading. This is a straightforward heading-hold implementation requiring no separate mode -- it is always active when armed.

4. Final Performance Verification

All three required scenarios were run against the final build with seed 42.

Hover (§9.1)

Roll and pitch remain within $\pm 1.5^\circ$ for the full 30-second run after the initial settling window. Yaw drifts at approximately $2^\circ/\text{min}$, inside the $\pm 5^\circ/\text{min}$ limit. Altitude climbs from the 1m starting position and locks at the 10m target from approximately $t = 8\text{s}$ onward, with no subsequent drift. **Pass.**

Tilt Recovery (§9.2)

Roll decays smoothly from 30° with no oscillation and no overshoot. It crosses into the $\pm 2^\circ$ band at approximately $t = 2.5\text{s}$. The spec requires this within 2.0s, so this scenario is narrowly outside the requirement. The response shape is correct -- well-damped, monotonic decay -- so the remaining gap is purely a matter of the outer angle gain being slightly conservative. Overshoot is under 1° . **Borderline -- narrowly outside the 2.0s target.**

Disturbance Rejection (§9.3)

The $0.1\text{ N}\cdot\text{m}$ roll impulse at $t = 2.0\text{s}$ causes a brief spike peaking at approximately 5° yaw and under 1° roll. All axes return inside the $\pm 2^\circ$ band within approximately 0.6s of the impulse ending. No sustained oscillation is visible after $t = 3\text{s}$. **Pass.**

5. Stretch Goal Summary

Goal	Status
Barometer mock driver	Complete — `baro_mock.c` implements `baro_dev_t`
1D Kalman altitude estimator	Complete — `altitude.c` with tunable Q/R parameters
ALT_HOLD: altitude → climb rate → throttle	Complete — active when throttle stick centered
Yaw heading hold	Complete — integrated yaw stick, always active when armed
Real-time 3D viewer	Not implemented

6. What Remains

The tilt recovery timing is the one unmet primary requirement. The fix is a small increase to

`kp_roll_angle` -- the response is already well-damped, so more outer-loop gain will speed the correction without introducing oscillation. I did not push this further because previous iterations showed that increasing the angle gain had side-effects on yaw coupling in the disturbance scenario, and I chose to leave the system at its best overall balance rather than optimise one metric at the expense of another.

The 3D viewer stretch goal (raylib or SDL) was not attempted. The remaining three stretch goals are complete.